



Projeto de Computação Gráfica: Space Invaders 3D

Este projeto visa recriar o clássico jogo *Space Invaders* em 3D, utilizando os conhecimentos adquiridos em Computação Gráfica.

Joana Moreira N37238

Gustavo Pina N50072

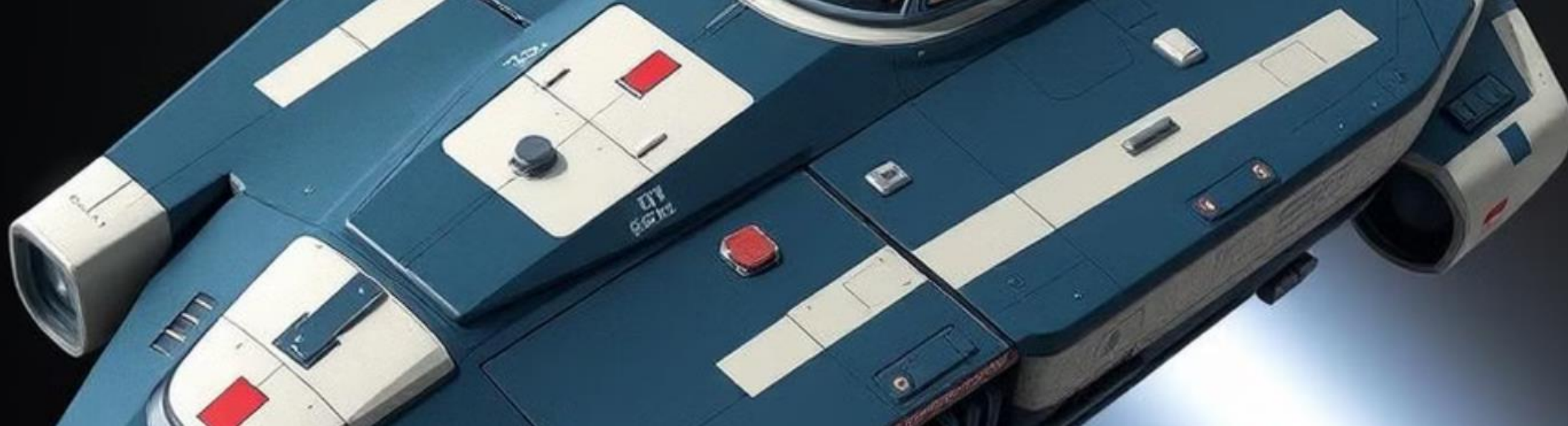
Introdução

Objetivo

Replicar o jogo clássico *Space Invaders* em 3D.

Desafios

Criar uma versão 3D do jogo, incluindo modelagem, iluminação, texturas e mecânicas de jogo.



Problema

1 Objetivo Principal

Criar um clone do jogo Space Invaders.

2 Desafios do Jogo

Como carregar várias figuras diferentes, as apresentar com o uso do GLEW e as fazer obedecer à lógica de jogo pretendida.

3 Regras do Jogo

Uma nave, que se deve mover horizontalmente enquanto naves inimigas a alvejam.

Solução

Modelagem

Criar modelos 3D para a nave do jogador, inimigos e cenário.

Mecânicas

Implementar movimentação horizontal da nave, disparos e padrões de movimento para os inimigos.

Realismo

Utilizar iluminação, materiais e texturas para criar um ambiente visualmente realista.





Implementação



Movimentação

Controlar a nave horizontalmente com as teclas do teclado.



Disparos

Disparar projéteis com a tecla "Espaço".



Colisão

Detectar colisões entre projéteis e naves ao usar *Hitboxes*.



Funcionalidades Adicionais

1

Renascer

As naves inimigas "renascem" após um tempo após serem destruídas.

2

Recomeçar

O jogador pode recomeçar o jogo ao pressionando a tecla "R".

3

Pontuação

O *Score* do jogador é apresentado no terminal após o fim do jogo.

4

Texturas

Texturas para melhorar a estética dos objetos e do ambiente de jogo.

Tecnologias

1

Modern OpenGL

Utilizado para renderização gráfica.

2

GLM

Biblioteca matemática para operações de vetores e matrizes.

3

GLFW

Biblioteca para criação de janelas e gestão de eventos.

4

GLEW

Biblioteca para carregar extensões OpenGL.

```
1 #include <iostream>
2 #include <GL/glew.h>
3 #include <GLFW/glfw3.h>
4 #include <glm/glm.hpp>
5 #include <glm/gtx/matrix_transform.hpp>
6 #include <glm/gtx/vec_swizzle.hpp>
7 #include <glm/gtx/quaternion.hpp>
8 #include <glm/gtx/epsilon.hpp>
9 #include <glm/gtx/intersect.hpp>
10 #include <glm/gtx/projection.hpp>
11 #include <glm/gtx/vec_swizzle.hpp>
12 #include <glm/gtx/quaternion.hpp>
13 #include <glm/gtx/epsilon.hpp>
14 #include <glm/gtx/intersect.hpp>
15 #include <glm/gtx/projection.hpp>
16 #include <glm/gtx/vec_swizzle.hpp>
17 #include <glm/gtx/quaternion.hpp>
18 #include <glm/gtx/epsilon.hpp>
19 #include <glm/gtx/intersect.hpp>
20 #include <glm/gtx/projection.hpp>
21 #include <glm/gtx/vec_swizzle.hpp>
22 #include <glm/gtx/quaternion.hpp>
23 #include <glm/gtx/epsilon.hpp>
24 #include <glm/gtx/intersect.hpp>
25 #include <glm/gtx/projection.hpp>
26 #include <glm/gtx/vec_swizzle.hpp>
27 #include <glm/gtx/quaternion.hpp>
28 #include <glm/gtx/epsilon.hpp>
29 #include <glm/gtx/intersect.hpp>
30 #include <glm/gtx/projection.hpp>
31 #include <glm/gtx/vec_swizzle.hpp>
32 #include <glm/gtx/quaternion.hpp>
33 #include <glm/gtx/epsilon.hpp>
34 #include <glm/gtx/intersect.hpp>
35 #include <glm/gtx/projection.hpp>
36 #include <glm/gtx/vec_swizzle.hpp>
37 #include <glm/gtx/quaternion.hpp>
38 #include <glm/gtx/epsilon.hpp>
39 #include <glm/gtx/intersect.hpp>
40 #include <glm/gtx/projection.hpp>
41 #include <glm/gtx/vec_swizzle.hpp>
42 #include <glm/gtx/quaternion.hpp>
43 #include <glm/gtx/epsilon.hpp>
44 #include <glm/gtx/intersect.hpp>
45 #include <glm/gtx/projection.hpp>
46 #include <glm/gtx/vec_swizzle.hpp>
47 #include <glm/gtx/quaternion.hpp>
48 #include <glm/gtx/epsilon.hpp>
49 #include <glm/gtx/intersect.hpp>
50 #include <glm/gtx/projection.hpp>
51 #include <glm/gtx/vec_swizzle.hpp>
52 #include <glm/gtx/quaternion.hpp>
53 #include <glm/gtx/epsilon.hpp>
54 #include <glm/gtx/intersect.hpp>
55 #include <glm/gtx/projection.hpp>
56 #include <glm/gtx/vec_swizzle.hpp>
57 #include <glm/gtx/quaternion.hpp>
58 #include <glm/gtx/epsilon.hpp>
59 #include <glm/gtx/intersect.hpp>
60 #include <glm/gtx/projection.hpp>
61 #include <glm/gtx/vec_swizzle.hpp>
62 #include <glm/gtx/quaternion.hpp>
63 #include <glm/gtx/epsilon.hpp>
64 #include <glm/gtx/intersect.hpp>
65 #include <glm/gtx/projection.hpp>
66 #include <glm/gtx/vec_swizzle.hpp>
67 #include <glm/gtx/quaternion.hpp>
68 #include <glm/gtx/epsilon.hpp>
69 #include <glm/gtx/intersect.hpp>
70 #include <glm/gtx/projection.hpp>
71 #include <glm/gtx/vec_swizzle.hpp>
72 #include <glm/gtx/quaternion.hpp>
73 #include <glm/gtx/epsilon.hpp>
74 #include <glm/gtx/intersect.hpp>
75 #include <glm/gtx/projection.hpp>
76 #include <glm/gtx/vec_swizzle.hpp>
77 #include <glm/gtx/quaternion.hpp>
78 #include <glm/gtx/epsilon.hpp>
79 #include <glm/gtx/intersect.hpp>
80 #include <glm/gtx/projection.hpp>
81 #include <glm/gtx/vec_swizzle.hpp>
82 #include <glm/gtx/quaternion.hpp>
83 #include <glm/gtx/epsilon.hpp>
84 #include <glm/gtx/intersect.hpp>
85 #include <glm/gtx/projection.hpp>
86 #include <glm/gtx/vec_swizzle.hpp>
87 #include <glm/gtx/quaternion.hpp>
88 #include <glm/gtx/epsilon.hpp>
89 #include <glm/gtx/intersect.hpp>
90 #include <glm/gtx/projection.hpp>
91 #include <glm/gtx/vec_swizzle.hpp>
92 #include <glm/gtx/quaternion.hpp>
93 #include <glm/gtx/epsilon.hpp>
94 #include <glm/gtx/intersect.hpp>
95 #include <glm/gtx/projection.hpp>
96 #include <glm/gtx/vec_swizzle.hpp>
97 #include <glm/gtx/quaternion.hpp>
98 #include <glm/gtx/epsilon.hpp>
99 #include <glm/gtx/intersect.hpp>
100 #include <glm/gtx/projection.hpp>
```

Conclusão

